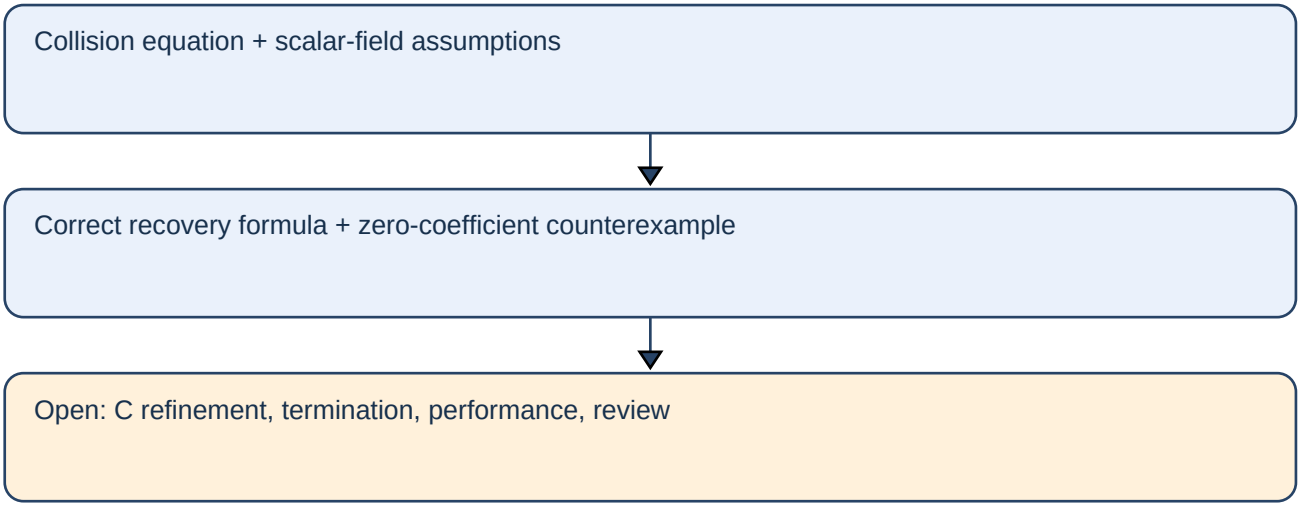
cryptanalysis: checked statements and open bridges



Logical obligations only. No curve route or measured performance is represented.

Lean retrospective, 2026-10-18

Status: **partial retrospective; four new theorems machine checked**. Independent semantic-fidelity review is pending. This is this repository's first research Lake project; the previous Lean source was the proof-gate audit.

Scope and source

The request covers past claims in crypto, cryptanalysis, and autoresearcher. This batch reviews the two collision equations in src/rho.c, rho_try_solve, at baseline commit `1f04dfdff100e7df5ca815b7ed5ba269bcf35f26`. inventory.json records every pre-existing Lean source path and Lake project at that revision. It is not a complete extraction of prose claims or a verification of the C implementation.

Claim	Lean declaration	Exact scope
Ordinary collision recovery	`Retrospective.Rho.recover`	Any field; b1 differs from b2
Negation-collision recovery	`Retrospective.Rho.recover_negated`	Any field; b1+b2 is nonzero
Group collision implies scalar recovery	`Retrospective.Rho.group_collision_recover`	Prime p, ZMod p coefficients, and an explicitly injective additive scalar encoding
Dropping the denominator condition is unsound	`Retrospective.Rho.zero_coefficient_not_injective`	Kernel-checked counterexample over ZMod 7

Readable argument and limits

An ordinary collision gives `a1+b1*x = a2+b2*x`, hence `(b1-b2)*x = a2-a1`. The nonzero coefficient is invertible in the scalar field, giving `x=(a2-a1)/(b1-b2)`. A negation collision instead gives `(b1+b2)*x=-(a1+a2)` and the corresponding formula with that denominator. Injectivity of the supplied group encoding lifts equality of actual group elements to the coefficient equation. The zero-coefficient map over ZMod 7 identifies 0 and 1, disproving an unconditional uniqueness assertion.

Primality and the scalar-encoding hypothesis are explicit. The source's composite-order gcd branch, bounded candidate enumeration, modular C routines, walk distribution, termination, expected running time, and point serialization have not been proved. This proves the recovery mathematics in the stated model; it does not refine the C program into Lean or establish attack success.

Verification

Proof map, source, and verification.log bind the statements and evidence. Lean `v4.35.0`, CryptoLean commit `68117089795313bc2ffe3a3b5f72052d5fa7cb91`, and all transitive dependency revisions are pinned. Local dependency checkouts were reused through ignored symlinks at those exact revisions.

```
bash research/lean_retrospective_20261018/check.sh
```

The command builds the actual modules, prints each theorem's axioms, and runs the shared fail-closed transitive axiom audit. All four pass on standard Lean foundations, with zero compiler-trusted proofs. The repository-wide `lean-proof-gate.yml` automatically discovers this new Lake project. `lean4checker` is not installed; external fresh rechecking was not performed.

Reproduce the original path inventory with a fresh output filename:

```
git ls-tree -r 1f04dfdff100e7df5ca815b7ed5ba269bcf35f26 | node
research/lean_retrospective_20261018/inventory.mjs 1f04dfdff100e7df5ca815b7ed5ba269bcf35f26
/tmp/cryptanalysis-inventory.json
```

Coverage remaining

The full historical review, a bridge from exact generator order to the supplied scalar encoding, implementation refinement, and independent semantic review remain open. See the cross-repository problem record. No historical measurements or claim statuses were rewritten.

Diagram, editable data, and report PDF show the logical chain. The source README's rho performance table and `docs/papers/ecc2k130-blackwell/claims-map.md` were located as potentially affected presentations. No timings or performance claims change, and no benchmark graph or curve route needs new data.